Learning Objectives

- Find out about the unit and how things will work.
- Get familiar with Ed.

Before you start going through this mock lesson, please make sure you check out our short **preliminary "unit"** available to all the students coming to FIT1008/2085. That unit is meant to help you understand what you are expected to have mastered by the time you join us and how to set up the tools necessary for our lessons and assignments.

Weekly workload expectations

Every week:

- You should go over and understand the pre-reading lesson: **<2 hours**
- ***After you have completed the lesson***, you should attend *each* of the following in-class sessions:
  - Workshop session: **2 hours**
  - Applied session: **2 hours**

We emphasise that in-class activities are available only after you have completed the pre-reading lesson.

For all the assessments, you are expected to work **~6 hours a week** but this is very variable and depends on you. Make sure you *start working early and submit your work on time*.

In-class activities

As mentioned above, there are two types of in-class activities:

Workshops:

- Workshops to be given offline by a lecturer in a large lecture room and are supposed to be *complementary* to the pre-reading materials and act as a lecture with a flavor of active learning, where you will be involved in solving a number of code challenges together with your peers and your lecturer. They will also be recorded for those of you who cannot attend; the recordings will be made available on the same day but typically a bit later.

Applied classes:

- to be given by a TA (or multiple TAs) in a smaller room. Applied sessions should facilitate your understanding of the *theory and practical part* of the subject. They are ideal for asking all sorts of questions regarding the materials covered.

Once again, you must come to any of the in-class sessions after going over the pre-reading materials.

Getting familiar with the Moodle site

If you have not done so already:

1. Go to the Moodle site of this unit.
2. Review the Unit Information page at the respective Moodle site.
3. Check that you have access to the pre-reading videos in Week 1.

Despite having a Moodle page for the unit, **most of your learning will be done on Ed**.

Getting help

If you need help with the unit, we recommend you try this first.

- Ask your tutors and classmates during applied sessions.
- Go to a consultation. See times on Moodle.
- Ask on ed. There should be staff answering Monday-Friday.
- At the end of every workshop, our lecturer is happy to answer questions about the workshop.
- For admin-related questions, email the following address. (*Direct emails to staff will be ignored!)*
    - **fit1008.clayton-x@monash.edu (AU)**
    - **muhammad.fermipasha@monash.edu (MY)**

Please *don't send us direct emails*. Our lecturers and the admin team have piles of unanswered emails. If you don't want your request to be among them, send an email to one of the addresses above.

Getting familiar with ed threads

1. First, get familiar with the rules and guidelines of ed, if you haven't already done so.
2. Create a thread on the discussion board. If you do not have a question or anything related to the unit to share, you can simply make a post under "Social" (e.g. introduce yourself). In any case, *use the proper category*!
3. Finally, browse ed and post a reply on at least two threads.

Consider the function below.

Python

```python
def below(n: int) -> int:
    total = 0
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            total += i
    return total
```

Question 1 ✓ SUBMITTED

You are not yet familiar with the concept of time complexity of an algorithm. So instead of formally assessing the complexity of this Python function, estimate how many times line 5 is *asymptotically* executed given some input integer n? Give the tightest bound.

- A constant
- $\approx n$
- $\approx n \cdot \log n$
- $\approx \frac{n^2}{2}$ ✓
- $\approx \frac{n^3}{3}$
- $\approx 2^n$

Explanation

The outer `for` loop of the function iterates n times. The inner `for` loop iterates $1, 2, \ldots, n$ times, depending on varying i. This makes the total number of times line 5 is executed be equal exactly to $1 + 2 + \ldots + n = \frac{1+n}{2} \cdot n$ (see this).

Since line 5 is essentially almost the only operation performed in the function, we can say that the total number of elementary operations is roughly $\frac{1+n}{2} \cdot n$. As we can we will see in the future lessons, this indicates that the *time complexity* of this algorithm can be asymptotically seen as a function asymptotically growing as n^2.

Question 2 ✓ SUBMITTED

Run by hand the function below for $n = 4$. What does it output?

30

Answer is 30

Explanation

Execute the program step by step via Python Tutor.